

C++ STL

1. Vector (#include <vector>)

Use cases: Dynamic arrays, when you need random access

```
vector<int> v;
v.push_back(5);           // Add to end
v.pop_back();             // Remove from end
v.size();                 // Get size
v.empty();                // Check if empty
v[i];                     // Access element (no bounds check)
v.at(i);                  // Access with bounds check
v.front(); v.back();      // First/last element
v.insert(v.begin()+2, 10); // Insert at position
v.erase(v.begin()+1);     // Remove at position
v.clear();                 // Remove all
sort(v.begin(), v.end()); // Sort
```

2. Set (#include <set>)

Use cases: Unique elements, sorted order, BST operations

```
set<int> s;
s.insert(5);               // Insert (auto-sorted)
s.erase(5);               // Remove
s.find(5);                 // Returns iterator, s.end() if not found
s.count(5);                // Returns 1 if exists, 0 otherwise
s.lower_bound(3);          // First element >= 3
s.upper_bound(3);          // First element > 3
s.size(); s.empty();
// Iteration is in sorted order
for(auto it = s.begin(); it != s.end(); it++)
```

3. Map (#include <map>)

Use cases: Key-value pairs, dictionary, frequency counting

```
map<string, int> m;
m["apple"] = 5;            // Insert/update
m.insert({"banana", 3});
m.erase("apple");          // Remove by key
m.find("apple");           // Returns iterator
m.count("apple");          // Returns 1 if exists
m.size(); m.empty();
m.lower_bound("b");        // First key >= "b"
// Access creates key if doesn't exist, use find() to check existence
```

4. Unordered Set/Map (#include <unordered_set>,

#include <unordered_map>)

Use cases: When you don't need ordering, faster than set/map

```
unordered_set<int> us;
unordered_map<string, int> um;
// Same interface as set/map but no lower_bound/upper_bound
// Faster O(1) average case for insert/find/delete
```

5. Priority Queue (#include <queue>)

Use cases: Max/min heap, always need largest/smallest element

```
priority_queue<int> pq; // Max heap (default)
pq.push(5);             // Insert
pq.top();               // Get max element (don't remove)
pq.pop();               // Remove max element
pq.size(); pq.empty();

// Min heap
priority_queue<int, vector<int>, greater<int>> min_pq;
```

6. Queue (#include <queue>)

Use cases: FIFO operations, BFS

```
queue<int> q;
q.push(5);           // Add to back
q.pop();             // Remove from front
q.front();           // Get front element
q.back();            // Get back element
q.size(); q.empty();
```

7. Stack (#include <stack>)

Use cases: LIFO operations, DFS

```
stack<int> st;
st.push(5);          // Add to top
st.pop();            // Remove from top
st.top();            // Get top element
st.size(); st.empty();
```

8. Algorithm Functions (#include <algorithm>)

```
sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<int>()); // Descending

// Binary search on sorted ranges
binary_search(v.begin(), v.end(), 5);
lower_bound(v.begin(), v.end(), 5); // First >= 5
```

```
upper_bound(v.begin(), v.end(), 5); // First > 5

// Other useful algorithms
max(a, b); min(a, b);
swap(a, b);
reverse(v.begin(), v.end());
unique(v.begin(), v.end()); // Remove consecutive duplicates
```

9. Pair and Tuple

```
pair<int, string> p = {1, "hello"};
p.first; p.second;

tuple<int, string, double> t = {1, "hello", 3.14};
get<0>(t); get<1>(t); get<2>(t);
```

Quick Decision Guide:

- **Need sorted unique elements** → set
- **Need key-value pairs with sorting** → map
- **Fast lookup without ordering** → unordered_set/map
- **Always need max/min element** → priority_queue
- **FIFO operations** → queue
- **LIFO operations** → stack
- **Dynamic array with random access** → vector
- **Small fixed size** → array

Common Patterns for Your Topics:

BST operations → Use set or map **Binary Heap** → Use priority_queue **Hashing** → Use unordered_set or unordered_map **Sorting** → Use sort() from <algorithm>